

Question
EF-Q1
Description

Implement Repository Pattern (Two Phases)

Scenario

You are building a Console based **Library Book Management System** for a training institute.

Initially, the institute does **not want to use a database**. The application should run using **temporary in-memory storage**.

Later, management decides to **store the data permanently in SQL Server**.

The system should be designed in such a way that **switching the storage implementation from In-Memory → SQL Server requires minimal code changes**.

Your task is to implement the system using the **Repository Pattern** so that the **data access layer can be replaced easily** without affecting the rest of the application.

Phase 1 – Repository Without Database

Create an **ASP.NET Core MVC application** that manages books using **in-memory storage**.

1. Model

Create a model class named **Book**.

Properties

Property	Type
BookId	int
Title	string
Author	string
Price	decimal

2. Repository Interface

Create an interface named:

```
IBookRepository
```

Methods

```
GetAllBooks()  
GetBookById(int id)  
AddBook(Book book)  
DeleteBook(int id)
```

⚠ Important Rule

Do **NOT** store `List<Book>` inside the controller.

All data operations must go through the **repository interface**.

3. In-Memory Repository Implementation

Create a class named:

```
MemoryBookRepository
```

This class must **implement** `IBookRepository`.

Requirements

- Store books **in memory**
- You may use:
 - `Dictionary<int, Book>`
 - `Array`
 - `List<Book>`
- Any suitable data structure

Initial Data

Initialize the repository with **3 sample books**.

Example:

- Clean Code – Robert C. Martin
- Design Patterns – GoF
- Refactoring – Martin Fowler

4. Controller

Create a controller named:

```
BookController
```

Required Actions

Action	Purpose
List	Display all books
Details	Display a single book
Create	Add a new book
Delete	Remove a book

Dependency Injection Requirement

The controller must use **constructor injection**.

Example idea:

```
private readonly IBookRepository _repo;

public BookController(IBookRepository repo)
{
    _repo = repo;
}
```

Expected Result (Phase 1)

The application should run **without SQL Server** and support:

- List Books
- View Book Details
- Add Book
- Delete Book

⚠ Note

All data will be stored **in memory**.

When the application **restarts**, all books will be **lost**.

Phase 2 – Migrate Repository to SQL Server

Management now wants to **store the data permanently in SQL Server**.

You must extend the same project to support **Entity Framework Core**.

⚠ **Do NOT change controller logic.**

Only the **repository implementation** should change.

Step 1 – Create DbContext

Create a class:

```
LibraryDbContext
```

Add a DbSet:

```
DbSet<Book> Books
```

Step 2 – SQL Repository Implementation

Create another repository implementation:

```
SqlBookRepository
```

This class must implement:

```
IBookRepository
```

Use **Entity Framework Core** to perform:

- Retrieve all books
 - Retrieve a book by ID
 - Add a new book
 - Delete a book
-
-

Expected Result (Phase 2)

After implementing the SQL repository:

The application should now:

- Store books in **SQL Server**
- Use **Entity Framework Core**
- Work with the **same controllers and views**
- Require **only a repository implementation change**

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
public class DefaultSolution
```

```
{
```

```
    public class Book
```

```
    {
```

```
        public int BookId;
```

```
        public string Title;
```

```
public string Author;  
  
public decimal Price;  
  
}
```

```
public interface IBookRepository  
{  
  
    List<Book> GetAllBooks();  
  
    Book GetBookById(int id);  
  
    void AddBook(Book book);  
  
    void DeleteBook(int id);  
  
}
```

```
public class MemoryBookRepository : IBookRepository  
{  
  
    private Dictionary<int, Book> books = new Dictionary<int, Book>();  
  
    public MemoryBookRepository()  
    {  
  
        books[1] = new Book { BookId = 1, Title = "Clean Code", Author = "Robert C. Martin", Price = 500 };  
  
        books[2] = new Book { BookId = 2, Title = "Design Patterns", Author = "GoF", Price = 700 };  
  
        books[3] = new Book { BookId = 3, Title = "Refactoring", Author = "Martin Fowler", Price = 650 };  
  
    }  
  
    public List<Book> GetAllBooks()
```

```
{  
    return books.Values.ToList();  
}  
  
public Book GetBookById(int id)  
{  
    return books.ContainsKey(id) ? books[id] : null;  
}  
  
public void AddBook(Book book)  
{  
    int newId = books.Keys.Max() + 1;  
    book.BookId = newId;  
    books[newId] = book;  
}  
  
public void DeleteBook(int id)  
{  
    if (books.ContainsKey(id))  
        books.Remove(id);  
}  
}  
  
public static void Main()
```



```
{  
    IBookRepository repo = new MemoryBookRepository();  
  
    // List books  
    foreach (var b in repo.GetAllBooks())  
        Console.WriteLine(b.BookId + " " + b.Title);  
  
    // Add book  
    repo.AddBook(new Book { Title = "Domain Driven Design", Author = "Eric Evans", Price = 800 });  
  
    Console.WriteLine("After Adding:");  
  
    foreach (var b in repo.GetAllBooks())  
        Console.WriteLine(b.BookId + " " + b.Title);  
  
    // Delete book  
    repo.DeleteBook(2);  
  
    Console.WriteLine("After Deleting:");  
  
    foreach (var b in repo.GetAllBooks())  
        Console.WriteLine(b.BookId + " " + b.Title);  
}  
}
```